

Sistema de Gestão de Locações de Imóveis

RCP Data Imob — Documentação Completa

Disciplina: **Laboratório de Programação 3**

Professor: **Matheus Vanzan**

Integrantes:

Marcell Parra Araújo B. Silva

Rafael Vargas Carvalheira

Ruan Pablo Rodrigues

Março de 2026 — atualizado em Abril de 2026 (Entregável 7)

Contents

1	Apresentação do Projeto	4
2	Objetivo do Sistema	4
3	Escopo Funcional	4
3.1	Gestão de Imóveis com Categorias	4
3.2	Sistema de Locações com Controle de Status	5
3.3	Controle Financeiro com Dashboard Analítico	5
4	Funcionalidade Seleccionada para o Mobile	5
5	Arquitetura da Solução	6
5.1	Serviços e Portas	6
6	Stack Tecnológica	6
6.1	Backend	6
6.2	Frontend	6
6.3	Infraestrutura	7
7	Modelo de Dados	7
7.1	Diagrama de Relacionamentos	7
7.2	Detalhamento das Tabelas	8
7.2.1	Tabela: imoveis	8
7.2.2	Tabela: clientes	8
7.2.3	Tabela: locacoes	9
7.2.4	Tabela: financeiro	9
8	Estrutura do Backend	9
8.1	Organização do Projeto	10
8.2	Conexão com o Banco de Dados	10
8.3	Variáveis de Ambiente	10
9	API REST	10
9.1	Endpoint de Healthcheck	11
9.2	Endpoints de Imóveis	11
9.3	Endpoints de Clientes	11
9.4	Endpoints de Locações	11
9.5	Endpoints de Categorias	12
9.6	Endpoints Financeiros	12
9.7	Resumo Geral dos Endpoints	12
10	Banco de Dados e Docker (Entregável 3)	12
10.1	Status dos Requisitos	13
10.2	Configuração do docker-compose.yml	13
10.3	Testes CRUD via Postman	14
11	Frontend e Docker (Entregável 4)	14
11.1	Requisitos Atendidos	14
11.2	Telas Implementadas	14
11.3	Consumo da API	15
12	Capturas de Tela do Frontend (Entregável 4)	15
12.1	Dashboard	15
12.2	Imóveis	15

12.3	Clientes	16
12.4	Locações	16
12.5	Financeiro	17
13	Funcionalidade F1 — Busca de Disponibilidade e Criação de Reserva (Entregável 5)	17
13.1	Descrição	17
13.2	Requisitos Atendidos	18
13.3	Endpoint Implementado	18
13.4	Fluxo Completo da F1	18
13.5	Arquivos Criados/Modificados	19
13.6	Testes Realizados	19
14	Funcionalidade F2 — Sistema de Reservas com Status (Entregável 6)	19
14.1	Visão Geral	20
14.2	Alterações no Banco de Dados	20
14.2.1	Migração aplicada à tabela locacoes	20
14.2.2	Regras de Consistência	21
14.3	Alterações no Backend	21
14.3.1	Novos Endpoints em /locacoes	21
14.3.2	Lógica de Verificação de Conflito	21
14.3.3	Cálculo Automático de Valor Total	21
14.4	Alterações no Frontend	22
14.4.1	Tela de Locações — Atualizada	22
14.4.2	Tela de Agenda — Nova (/calendario)	22
14.4.3	Arquivos Modificados/Criados	23
14.5	Fluxo Completo da F2	23
14.6	Resumo dos Endpoints da F2	24
14.7	Instruções de Instalação e Atualização	25
15	Funcionalidade F3 — Controle Financeiro com Dashboard Analítico (Entregável 7)	25
15.1	Visão Geral	26
15.2	Requisitos Atendidos	26
15.3	Alterações no Banco de Dados	26
15.3.1	Migração aplicada à tabela financeiro	26
15.4	Alterações no Backend (backend/src/routes/financeiro.js)	27
15.4.1	Endpoints novos e modificados	27
15.4.2	Auto-update de atrasados	27
15.4.3	Exemplo de resposta — GET /financeiro/resumo	27
15.4.4	Exemplo de resposta — GET /financeiro/por-mes?meses=6	28
15.5	Alterações no Frontend	28
15.5.1	Tela Financeiro — Reescrita (/financeiro)	28
15.5.2	Dashboard — Atualizado	29
15.5.3	Arquivos Criados/Modificados	29
15.6	Fluxo Completo da F3 (end-to-end no Docker)	30
15.7	Resumo dos Endpoints F3	30
16	Execução do Ambiente	31
16.1	Acessos	31
16.2	Telas Disponíveis	31

17 Considerações sobre a Implementação	31
18 Conclusão	32
19 Referências Internas do Projeto	33

1. Apresentação do Projeto

O projeto Sistema de Gestão de Locações de Imóveis consiste em uma aplicação web desenvolvida para apoiar a administração de imóveis destinados à locação. A proposta central do sistema é reunir, em uma única plataforma, as operações de cadastro de imóveis e clientes, controle de reservas, verificação de disponibilidade e acompanhamento financeiro da operação.

A solução foi concebida em arquitetura cliente-servidor desacoplada, com backend em API REST e frontend em SPA (Single Page Application). Essa separação facilita a manutenção do sistema, melhora a organização do código e permite reutilização futura da API por aplicações mobile.

Além disso, o projeto foi estruturado para atender aos requisitos práticos da disciplina, contemplando desde a definição do tema e da stack tecnológica até a implementação de serviços executáveis em ambiente containerizado e a disponibilização de endpoints REST organizados por entidade de negócio.

2. Objetivo do Sistema

O objetivo do sistema é oferecer uma solução completa para o gerenciamento de locações de imóveis, permitindo:

- Cadastrar e manter o portfólio de imóveis disponíveis;
- Cadastrar clientes e seus dados de contato;
- Criar, consultar, editar e cancelar locações;
- Verificar disponibilidade de imóveis com base em período e restrições de data;
- Categorizar imóveis por características relevantes;
- Registrar receitas e despesas associadas à operação;
- Acompanhar indicadores financeiros e operacionais.

Com isso, o sistema busca tornar o processo de administração de locações mais organizado, rastreável e eficiente.

3. Escopo Funcional

O sistema foi planejado em torno de três módulos principais, detalhados a seguir.

3.1 Gestão de Imóveis com Categorias

Módulo responsável pelo cadastro e manutenção dos imóveis disponíveis para locação.

- Cadastro completo de imóveis com informações como título, descrição, endereço, cidade, estado, CEP, número de quartos, banheiros, vagas de garagem, área, valor de aluguel e valor de venda;
- Associação de múltiplas categorias por imóvel (Piscina, Ar-condicionado, Pet-friendly, etc.);

- Criação de novas categorias durante o fluxo de cadastro;
- Busca e filtros por título, cidade e status de disponibilidade;
- Visualização em cards com informações resumidas;
- Edição e exclusão com validações de integridade.

3.2 Sistema de Locações com Controle de Status

Módulo central da aplicação, responsável pelo controle das locações.

- Listagem de locações com imóvel, cliente, datas, valor mensal e status (ativa/inativa);
- Formulário de nova locação com selects dinâmicos de imóvel e cliente;
- Encerramento de locações ativas via PATCH;
- Indicação visual de locação ativa vs. inativa com badges semânticos;
- Cálculo e exibição de valor mensal em formato de moeda brasileira.

3.3 Controle Financeiro com Dashboard Analítico

Módulo que consolida receitas e despesas associadas à operação.

- Registro de lançamentos financeiros com tipo (receita/despesa), valor, data de vencimento e locação associada;
- Filtros por status (pendente/pago/atrasado);
- Cards de resumo com total de receitas e total de despesas;
- Registro de pagamento via PATCH;
- Dashboard com KPIs consolidados e gráfico de evolução financeira dos últimos 6 meses;
- Listagem das últimas locações registradas.

4. Funcionalidade Seleccionada para o Mobile

A funcionalidade escolhida para adaptação em ambiente mobile foi a Busca de Disponibilidade e Criação de Reserva. Essa escolha foi motivada por se tratar de um fluxo frequente para administradores de imóveis, especialmente em situações fora do escritório.

Recursos previstos para o mobile:

- Tela com campos de data de entrada e saída para busca rápida;
- Listagem de imóveis disponíveis no período informado;
- Exibição resumida com tipo, capacidade e valor estimado;
- Tela de detalhes do imóvel com categorias;
- Confirmação de reserva com seleção de cliente;
- Feedback imediato em caso de sucesso ou conflito de datas.

Justificativa: A consulta de disponibilidade e a confirmação de reservas são tarefas que exigem agilidade. Uma versão mobile desse fluxo reduz a dependência de computador e acelera o atendimento ao cliente.

5. Arquitetura da Solução

A aplicação adota uma arquitetura dividida em três camadas principais:

- **Banco de dados:** responsável pelo armazenamento persistente dos dados do sistema;
- **Backend:** responsável pela lógica de negócio, exposição dos endpoints e comunicação com o banco;
- **Frontend:** responsável pela interface visual e interação com o usuário.

Os serviços são orquestrados com Docker Compose, o que simplifica a execução do ambiente de desenvolvimento e padroniza a configuração entre diferentes máquinas.

5.1 Serviços e Portas

Serviço	Imagem / Tecnologia	Porta	Descrição
db	postgres:13-alpine	5432	Banco de dados PostgreSQL
backend	node:18-alpine (build)	3000	API REST Node.js / Express
frontend	node:20-alpine (build)	5173	Interface React / Vite

Table 1: *Serviços e portas do ambiente Docker*

6. Stack Tecnológica

6.1 Backend

Tecnologia	Versão	Função no projeto
Node.js	v18+	Runtime JavaScript no servidor
Express.js	4.18.2	Framework para API REST
PostgreSQL	v13+	Banco de dados relacional principal
pg	8.11.3	Driver PostgreSQL para Node.js
cors	2.8.5	Middleware para Cross-Origin Requests

Table 2: *Stack do backend*

6.2 Frontend

Tecnologia	Versão	Função no projeto
React	v18.2	Construção da interface SPA
Vite	v6.0	Build tool e servidor de desenvolvimento
React Router DOM	v7.0	Roteamento client-side
Tailwind CSS	v3.4	Estilização responsiva com utility-first
Recharts	v2.12	Gráficos do dashboard financeiro
Lucide React	v0.400	Biblioteca de ícones
Axios	v1.7	Comunicação HTTP com a API

Table 3: *Stack do frontend*

6.3 Infraestrutura

Tecnologia	Função
Docker	Containerização dos serviços
Docker Compose	Orquestração multi-container
Google Fonts	Tipografia (Plus Jakarta Sans + DM Sans)

Table 4: *Infraestrutura*

7. Modelo de Dados

As tabelas principais do sistema estão organizadas conforme a estrutura abaixo.

Tabela	Conteúdo
usuarios	Dados de acesso e perfil dos administradores
imoveis	Cadastro dos imóveis com atributos, valores e status
categorias_imoveis	Categorias personalizadas para classificação
imovel_categorias	Relacionamento N:N entre imóveis e categorias
clientes	Cadastro de locatários
locacoes	Contratos de locação, datas, valores e status
financeiro	Lançamentos de receitas e despesas

Table 5: *Tabelas do sistema*

7.1 Diagrama de Relacionamentos

```

categorias_imoveis  --(N:N)--  imovel_categorias  --(N:N)--  imoveis
                                                                |
                                                                (1:N)
                                                                |
                                locacoes  --(1:N)--  financeiro
                                |
                                imoveis
                                |
                                (N:1)
                                |
                                clientes

```

7.2 Detalhamento das Tabelas

7.2.1 Tabela: imoveis

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
titulo	VARCHAR(255)	NOT NULL	Título do imóvel
descricao	TEXT	—	Descrição detalhada
endereco	VARCHAR(500)	NOT NULL	Endereço completo
cidade	VARCHAR(100)	—	Cidade
estado	VARCHAR(2)	—	UF
cep	VARCHAR(10)	—	CEP
valor_aluguel	NUMERIC(12,2)	—	Valor mensal de aluguel
valor_venda	NUMERIC(12,2)	—	Valor de venda
area	NUMERIC(10,2)	—	Área em m ²
quartos	INTEGER	DEFAULT 0	Número de quartos
banheiros	INTEGER	DEFAULT 0	Número de banheiros
vagas_garagem	INTEGER	DEFAULT 0	Vagas de garagem
disponivel	BOOLEAN	DEFAULT TRUE	Disponível para locação
criado_em	TIMESTAMP	DEFAULT NOW()	Data de cadastro

Table 6: Estrutura da tabela imoveis

7.2.2 Tabela: clientes

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
nome	VARCHAR(255)	NOT NULL	Nome completo
cpf	VARCHAR(14)	UNIQUE	CPF (000.000.000-00)
email	VARCHAR(255)	—	E-mail de contato
telefone	VARCHAR(20)	—	Telefone
endereco	VARCHAR(500)	—	Endereço do cliente
criado_em	TIMESTAMP	DEFAULT NOW()	Data de cadastro

Table 7: Estrutura da tabela clientes

7.2.3 Tabela: locacoes

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
imovel_id	INTEGER	FK → imoveis(id)	Imóvel locado
cliente_id	INTEGER	FK → clientes(id)	Locatário
data_inicio	DATE	NOT NULL	Início do contrato
data_fim	DATE	—	Fim do contrato
valor_mensal	NUMERIC(12,2)	NOT NULL	Valor mensal
status	VARCHAR(50)	DEFAULT 'pendente'	pendente confirmada cancelada
valor_total	NUMERIC(12,2)	—	Valor total calculado
ativa	BOOLEAN	DEFAULT TRUE	Locação em vigor
criado_em	TIMESTAMP	DEFAULT NOW()	Data de registro

Table 8: Estrutura da tabela locacoes

7.2.4 Tabela: financeiro

Coluna	Tipo	Restrições	Descrição
id	SERIAL	PRIMARY KEY	Identificador único
locacao_id	INTEGER	FK → locacoes(id)	Locação relacionada
tipo	VARCHAR(50)	NOT NULL	receita / despesa
valor	NUMERIC(12,2)	NOT NULL	Valor do lançamento
data_vencimento	DATE	NOT NULL	Data de vencimento
data_pagamento	DATE	—	Data de pagamento
status	VARCHAR(50)	DEFAULT 'pendente'	pendente / pago / atrasado
criado_em	TIMESTAMP	DEFAULT NOW()	Data de registro

Table 9: Estrutura da tabela financeiro

8. Estrutura do Backend

O backend foi organizado de forma modular, com separação entre ponto de entrada, acesso ao banco e rotas por entidade.

8.1 Organização do Projeto

Caminho	Descrição
backend/src/index.js	Ponto de entrada; inicializa o Express e registra as rotas
backend/src/db/pool.js	Pool de conexão com PostgreSQL
backend/src/routes/imoveis.js	Rotas CRUD de imóveis + disponibilidade (F1)
backend/src/routes/clientes.js	Rotas CRUD de clientes
backend/src/routes/locacoes.js	Rotas CRUD de locações
backend/src/routes/categorias.js	Rotas CRUD de categorias
backend/src/routes/financeiro.js	Rotas CRUD de lançamentos financeiros
backend/src/routes/status.js	Endpoint de healthcheck

Table 10: Organização dos arquivos do backend

8.2 Conexão com o Banco de Dados

A comunicação com o PostgreSQL é realizada por meio de um pool de conexões (`pg.Pool`). Esse mecanismo evita a abertura e o fechamento de uma nova conexão a cada requisição, reduzindo overhead e melhorando a eficiência do backend.

```
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  database: process.env.DB_NAME || 'rcp_data_imob',
  user: process.env.DB_USER || 'admin',
  password: process.env.DB_PASSWORD || 'admin123',
});
```

8.3 Variáveis de Ambiente

Variável	Valor padrão	Descrição
DB_HOST	db	Host do serviço PostgreSQL no Compose
DB_PORT	5432	Porta do PostgreSQL
DB_NAME	rcp_data_imob	Nome do banco de dados
DB_USER	admin	Usuário de acesso
DB_PASSWORD	admin123	Senha de acesso

Table 11: Variáveis de ambiente do backend

9. API REST

A API foi construída com Node.js e Express, seguindo princípios REST para organização de recursos, uso semântico dos verbos HTTP e respostas padronizadas em JSON. O servidor escuta na porta 3000, e as rotas estão separadas por entidade.

9.1 Endpoint de Healthcheck

Método	Rota	Descrição	Resposta
GET	/status	Verifica status da aplicação e do banco	<code>{"status": "ok", "banco": "conectado"}</code>

9.2 Endpoints de Imóveis

Método	Rota	Descrição
GET	/imoveis	Lista todos os imóveis cadastrados
GET	/imoveis/disponibilidade	Lista imóveis livres em período (F1) com categorias
GET	/imoveis/:id	Retorna um imóvel específico pelo ID
POST	/imoveis	Cria um novo imóvel
PUT	/imoveis/:id	Atualiza os dados de um imóvel
DELETE	/imoveis/:id	Remove um imóvel do sistema

Table 12: *Endpoints de imóveis*

9.3 Endpoints de Clientes

Método	Rota	Descrição
GET	/clientes	Lista todos os clientes cadastrados
GET	/clientes/:id	Retorna um cliente específico pelo ID
POST	/clientes	Cadastra um novo cliente
PUT	/clientes/:id	Atualiza os dados de um cliente
DELETE	/clientes/:id	Remove um cliente do sistema

Table 13: *Endpoints de clientes*

9.4 Endpoints de Locações

Método	Rota	Descrição
GET	/locacoes	Lista todas as locações (aceita ?status=)
GET	/locacoes/:id	Retorna uma locação específica
POST	/locacoes	Cria um novo contrato de locação
PUT	/locacoes/:id	Edição completa com revalidação de disponibilidade
PATCH	/locacoes/:id/status	Altera apenas o status, revalidando conflito ao confirmar
PATCH	/locacoes/:id/encerrar	Encerra uma locação ativa (status=cancelada)

Table 14: *Endpoints de locações*

9.5 Endpoints de Categorias

Método	Rota	Descrição
GET	/categorias	Lista todas as categorias disponíveis
POST	/categorias	Cria uma nova categoria
DELETE	/categorias/:id	Remove uma categoria do sistema

Table 15: *Endpoints de categorias*

9.6 Endpoints Financeiros

Método	Rota	Descrição
GET	/financeiro	Lista lançamentos com filtros (?tipo=&status=&locacao_id=&data_inicio=&data_fim=)
GET	/financeiro/resumo	KPIs analíticos: receitas, despesas, saldo, pendentes, atrasados (F3)
GET	/financeiro/por-mes	Série temporal agregada (?meses=N) para gráficos (F3)
GET	/financeiro/:id	Detalhe de um lançamento por ID (F3)
POST	/financeiro	Registra um novo lançamento (aceita descricao)
PUT	/financeiro/:id	Edição completa do lançamento (F3)
PATCH	/financeiro/:id/pagar	Registra pagamento de um lançamento
DELETE	/financeiro/:id	Remove o lançamento (F3)

Table 16: *Endpoints financeiros (atualizados pela F3)*

9.7 Resumo Geral dos Endpoints

Entidade	GET	POST	PUT	PATCH	DELETE	Total
/status	1	—	—	—	—	1
/imoveis	3	1	1	—	1	6
/clientes	2	1	1	—	1	5
/locacoes	2	1	1	2	—	6
/categorias	1	1	—	—	1	3
/financeiro	4	1	1	1	1	8
TOTAL	13	5	4	3	4	29

Table 17: *Resumo geral dos endpoints da API*

10. Banco de Dados e Docker (Entregável 3)

O Entregável 3 consolidou a configuração do banco de dados PostgreSQL via Docker, a modelagem das entidades do sistema e a integração completa entre o backend Node.js/Express e o banco de dados. O ambiente é orquestrado pelo Docker Compose, garantindo portabilidade e facilidade de execução em qualquer máquina.

10.1 Status dos Requisitos

Requisito	Status
Banco de dados configurado no Docker (PostgreSQL 13)	Concluído
Modelagem das entidades e relacionamentos	Concluído
Backend integrado ao banco com persistência via API	Concluído
CRUD completo testado via Postman	Concluído
Documentação e capturas de tela	Concluído

Table 18: *Status dos requisitos — Entregável 3*

10.2 Configuração do docker-compose.yml

```
services:
  db:
    image: postgres:13-alpine
    environment:
      POSTGRES_DB: rcp_data_imob
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: rcp_data_imob
      DB_USER: admin
      DB_PASSWORD: admin123
  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    environment:
      - VITE_API_URL=http://localhost:3000
volumes:
  pgdata:
```

10.3 Testes CRUD via Postman

Operação	Método	Endpoint	Status
Create	POST	/clientes	201 Created
Read (listar)	GET	/clientes	200 OK
Update	PUT	/clientes/:id	200 OK
Delete	DELETE	/clientes/:id	200 OK

Table 19: *Testes CRUD via Postman*

11. Frontend e Docker (Entregável 4)

O Entregável 4 implementou o frontend completo do sistema com React, configurado e executável via Docker, consumindo dados reais da API REST do backend.

11.1 Requisitos Atendidos

Requisito	Status
Frontend configurado e rodando no Docker	Concluído
Tela inicial do sistema criada (Dashboard)	Concluído
Navegação mínima entre telas (menu/rotas)	Concluído
Frontend consumindo dados da API (múltiplas chamadas)	Concluído
Exibição de dados vindos da API (listas, KPIs, gráficos)	Concluído
Documentação correspondente e capturas de tela	Concluído

Table 20: *Status dos requisitos — Entregável 4*

11.2 Telas Implementadas

Rota	Tela	Funcionalidades
/	Dashboard	4 KPIs, gráfico de barras (Recharts) com 6 meses, últimas locações
/imoveis	Imóveis	Cards com filtros (cidade/status/busca), CRUD completo via modal
/clientes	Clientes	Tabela com busca por nome, CRUD completo via modal
/locacoes	Locações	Tabela com selects de imóvel/cliente, encerramento de locação
/financeiro	Financeiro	Cards receitas/despesas, filtro por status, ação de pagamento

Table 21: *Telas implementadas no frontend*

11.3 Consumo da API

Tela	Endpoints consumidos
Dashboard	GET /imoveis, GET /clientes, GET /locacoes, GET /financeiro
Imóveis	GET /imoveis, POST /imoveis, PUT /imoveis/:id, DELETE /imoveis/:id
Clientes	GET /clientes, POST /clientes, PUT /clientes/:id, DELETE /clientes/:id
Locações	GET /locacoes, GET /imoveis, GET /clientes, POST /locacoes, PATCH /locacoes/:id/encerrar
Financeiro	GET /financeiro, GET /locacoes, POST /financeiro, PATCH /financeiro/:id/pagar

Table 22: Consumo da API por tela

12. Capturas de Tela do Frontend (Entregável 4)

A seguir são apresentadas as capturas de tela das 5 telas principais do sistema, documentando o estado visual de cada módulo implementado.

12.1 Dashboard

Tela inicial com visão geral do sistema: 4 cards de KPIs (Total de Imóveis, Total de Clientes, Locações Ativas, Receita do Mês), gráfico de evolução financeira dos últimos 6 meses (Recharts) e lista das últimas locações.

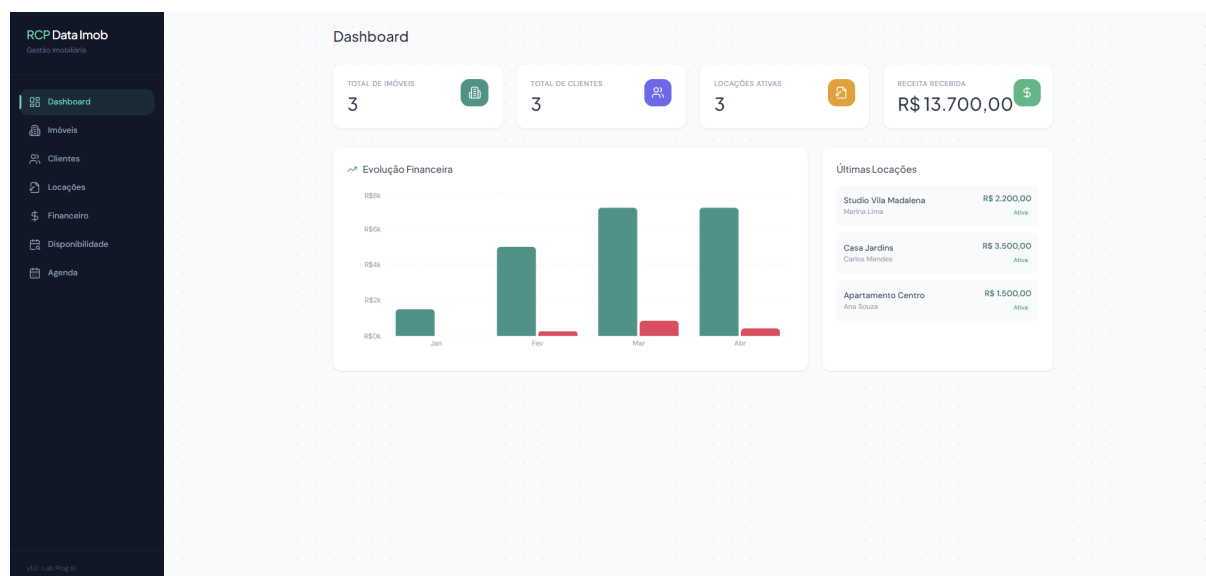


Figure 1: Dashboard com KPIs, gráfico de evolução financeira e últimas locações

12.2 Imóveis

Tela de gestão de imóveis com barra de busca por título, filtros por cidade e status de disponibilidade, e botão “Novo Imóvel” para cadastro. Os imóveis são exibidos em cards com informações resumidas.

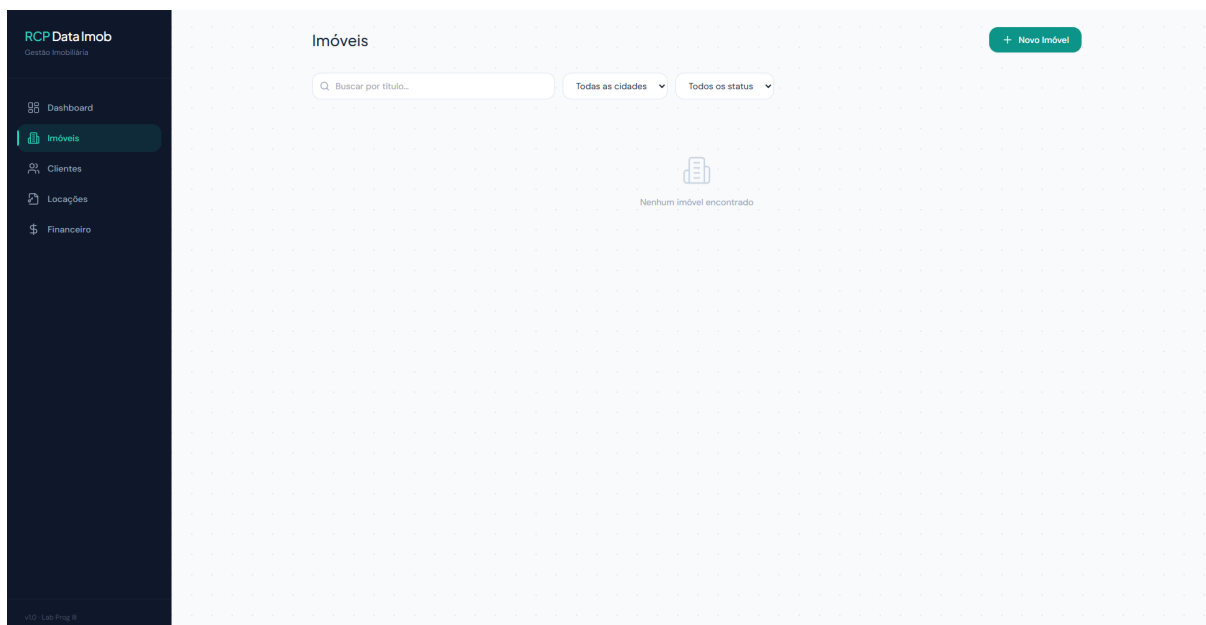


Figure 2: Listagem de imóveis com filtros e busca

12.3 Clientes

Tela de gestão de clientes com busca por nome e botão “Novo Cliente”. Os dados são apresentados em tabela com colunas Nome, CPF, Email, Telefone e ações de edição/exclusão.

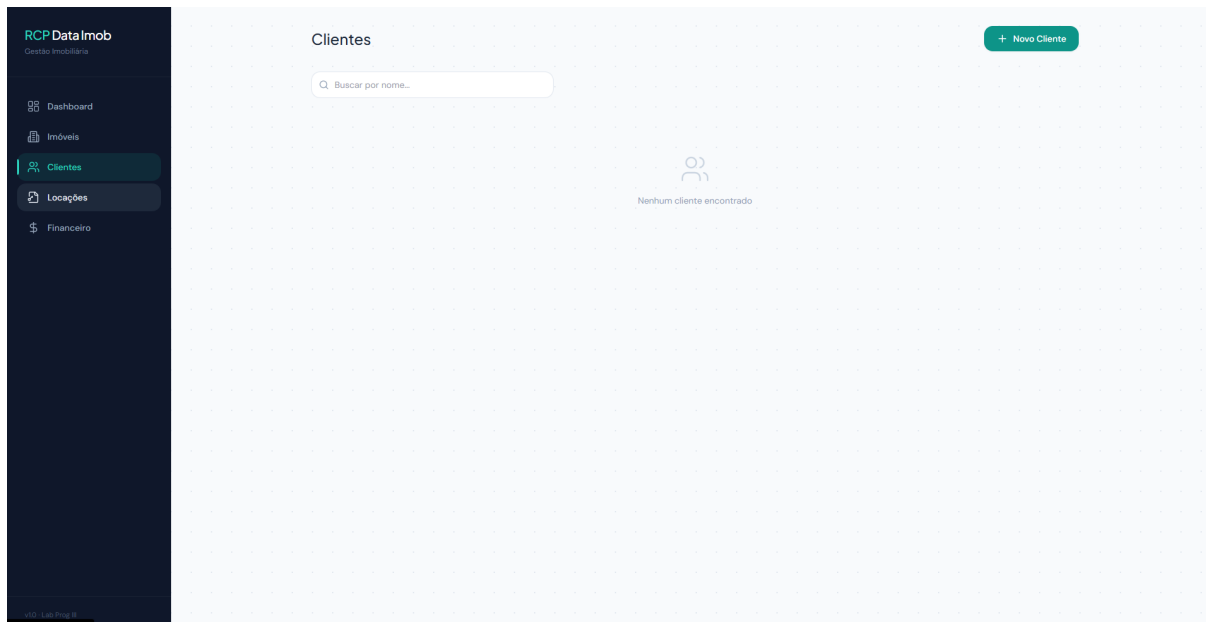


Figure 3: Listagem de clientes com busca por nome

12.4 Locações

Tela de gestão de locações com listagem de contratos contendo imóvel, cliente, datas, valor mensal e status (ativo/inativo). Botão “Nova Locação” para registro com selects dinâmicos.

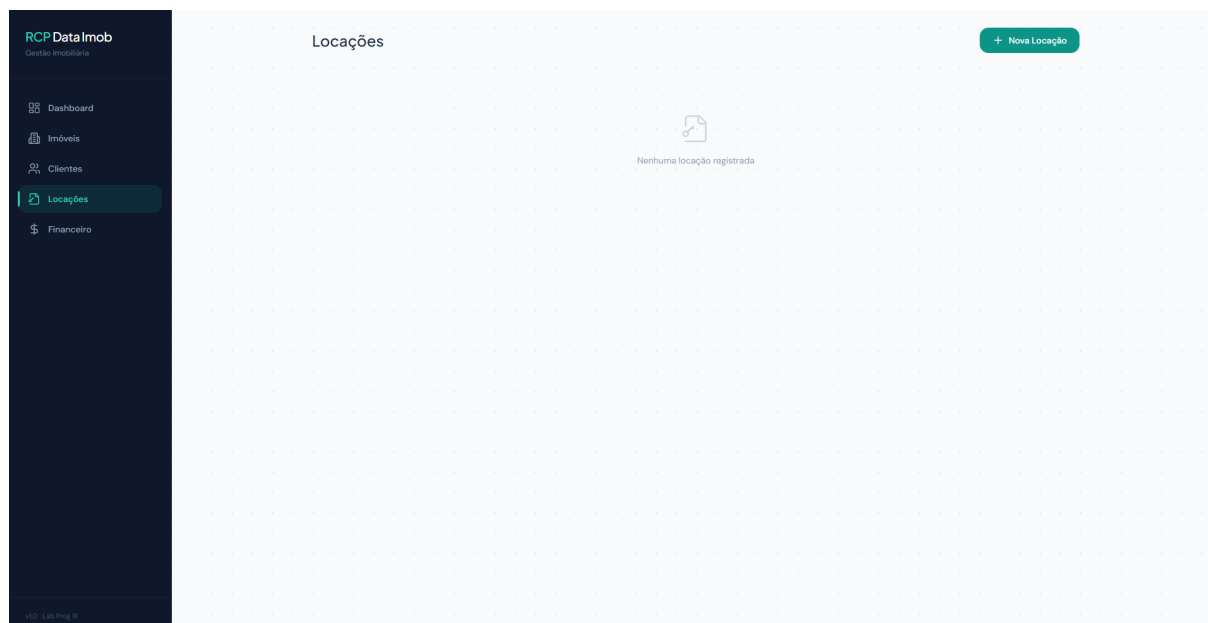


Figure 4: Listagem de locações com status e ação de encerramento

12.5 Financeiro

Tela financeira com cards de resumo (Total Receitas e Total Despesas), filtro por status (pendente/pago/atrasado), e tabela de lançamentos com ação de pagamento.

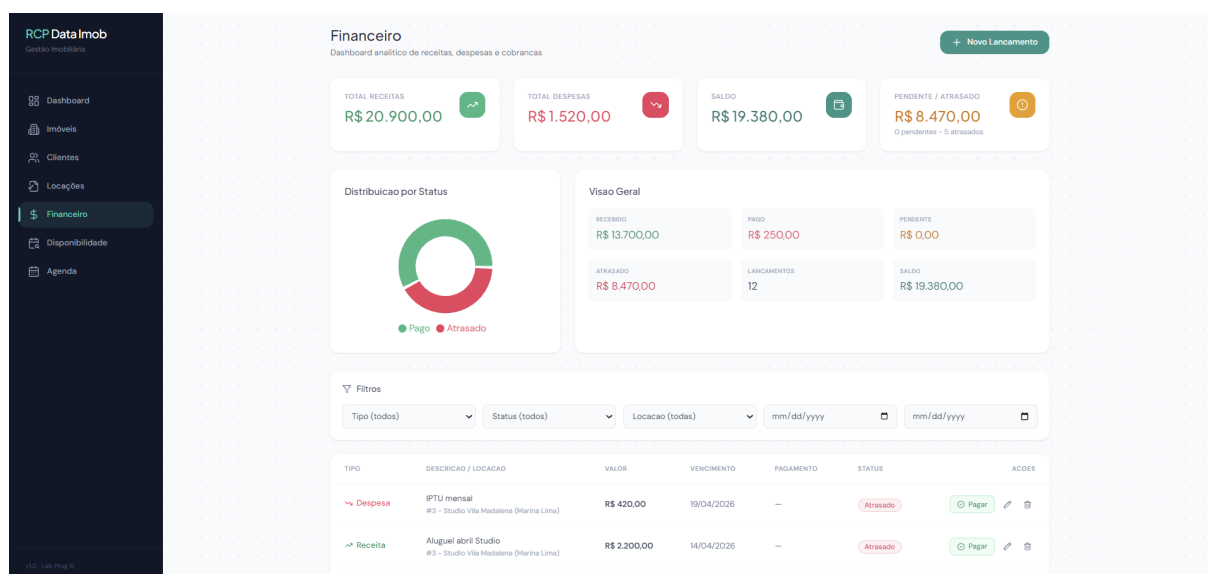


Figure 5: Painel financeiro com resumo e filtros por status

13. Funcionalidade F1 — Busca de Disponibilidade e Criação de Reserva (Entregável 5)

13.1 Descrição

O Entregável 5 implementou a Funcionalidade F1 completa: Busca de Disponibilidade e Criação de Reserva. Esta funcionalidade permite ao usuário buscar imóveis disponíveis

em um período específico e criar reservas diretamente pela interface web.

13.2 Requisitos Atendidos

Requisito	Status
Endpoint da API para busca de disponibilidade implementado	Concluído
Lógica de conflito de datas no banco	Concluído
Retorno de categorias do imóvel no endpoint	Concluído
Tela de busca com campos de período	Concluído
Listagem de imóveis disponíveis em cards	Concluído
Modal de confirmação de reserva com seleção de cliente	Concluído
Feedback de sucesso e atualização da listagem	Concluído
Fluxo completo funcionando no Docker	Concluído

Table 23: Status dos requisitos — Entregável 5 (F1)

13.3 Endpoint Implementado

Método	Rota	Parâmetros	Descrição
GET	/imoveis/disponibilidade	data_inicio, data_fim (query)	Lista imóveis disponíveis no período com categorias

Lógica de conflito de datas: um imóvel é considerado **INDISPONÍVEL** se existir uma locação ativa (`ativa = true`) cujo período se sobreponha ao período solicitado:

```
NOT EXISTS (
  SELECT 1 FROM locacoes l
  WHERE l.imovel_id = i.id
        AND l.ativa = true
        AND l.data_inicio <= data_fim
        AND (l.data_fim IS NULL OR l.data_fim >= data_inicio)
)
```

Isso garante que:

- Locações sem `data_fim` (abertas) bloqueiam o imóvel indefinidamente;
- Qualquer sobreposição parcial de período também bloqueia;
- Apenas locações ativas (`ativa = true`) são consideradas no bloqueio.

13.4 Fluxo Completo da F1

1. O usuário acessa a tela “Disponibilidade” via sidebar (ícone CalendarSearch);
2. Preenche Data de Entrada e Data de Saída e clica em “Buscar Disponíveis”;
3. O frontend chama `GET /imoveis/disponibilidade?data_inicio=&data_fim=;`

- Imóveis disponíveis são exibidos em cards com: título, cidade/estado, quartos, banheiros, vagas de garagem, área, categorias como badges coloridos e valor mensal;
- O usuário clica em “Reservar” em um imóvel;
- Modal de confirmação é exibido com: resumo do imóvel, período pré-preenchido (read-only), select de cliente, campo de valor mensal editável;
- Ao confirmar, o frontend chama `POST /locacoes` criando a locação no banco;
- Banner de sucesso é exibido e o imóvel é removido da listagem;
- Em caso de erro (ex: conflito), mensagem de erro aparece no modal.

13.5 Arquivos Criados/Modificados

Arquivo	Operação	Descrição
backend/src/routes/imoveis.js	Modificado	Adicionado endpoint GET /disponibilidade com lógica SQL de conflito
frontend/src/pages/Disponibilidade.jsx	Criado	Nova página com form de busca, cards e modal de reserva
frontend/src/App.jsx	Modificado	Rota /disponibilidade adicionada
frontend/src/components/Layout.jsx	Modificado	Item “Disponibilidade” com ícone CalendarSearch na sidebar

Table 24: Arquivos criados/modificados — Entregável 5

13.6 Testes Realizados

Teste	Esperado	Resultado
GET /disponibilidade sem params	HTTP 400	Passou
Busca em período livre (2 imóveis disponíveis)	2 imóveis	Passou
Busca em período com imóvel ocupado	1 imóvel (conflito)	Passou
Categorias retornadas no JSON	Array categorias	Passou
Criar reserva via POST /locacoes	Locação criada ativa=true	Passou
Busca após reserva (imóvel some)	1 imóvel removido	Passou
Endpoint via proxy Vite /api/...	JSON com categorias	Passou
Frontend acessível em localhost:5173	HTTP 200	Passou

Table 25: Testes realizados — Entregável 5

14. Funcionalidade F2 — Sistema de Reservas com Status (Entregável 6)

14.1 Visão Geral

O Entregável 6 implementa a Funcionalidade F2 — Sistema de Reservas com Verificação de Disponibilidade, expandindo o módulo de locações com controle completo de ciclo de vida de reservas. As principais adições são: campo **status** (Pendente / Confirmada / Cancelada) na tabela de locações, cálculo automático de valor total, edição de reservas existentes com revalidação de conflito de datas, e tela de Agenda com calendário mensal interativo.

Requisito	Status
Status de reserva (Pendente / Confirmada / Cancelada) no banco e na API	Concluído
Edição de reservas com revalidação de disponibilidade	Concluído
Cálculo automático de valor total com base no período	Concluído
Tela de Agenda — calendário mensal com cores por status	Concluído
Filtros por status na listagem de locações	Concluído
Fluxo completo funcionando no Docker	Concluído

Table 26: Status dos requisitos — Entregável 6 (F2)

14.2 Alterações no Banco de Dados

14.2.1 Migração aplicada à tabela locacoes

Foram adicionadas duas novas colunas à tabela existente via script de migração (`migration_f2.sql`), sem impacto nos dados já existentes:

Coluna	Tipo	Padrão	Descrição
status	VARCHAR(50)	'pendente'	Estado da reserva: pendente confirmada cancelada
valor_total	NUMERIC(12,2)	NULL	Valor calculado automaticamente com base no período

Table 27: Colunas adicionadas pela migração F2

```
ALTER TABLE locacoes
ADD COLUMN IF NOT EXISTS status VARCHAR(50) DEFAULT 'pendente',
ADD COLUMN IF NOT EXISTS valor_total NUMERIC(12,2);

-- Atualiza registros existentes
UPDATE locacoes
SET status = CASE WHEN ativa = true THEN 'confirmada' ELSE 'cancelada' END
WHERE status IS NULL OR status = 'pendente';
```

Como executar a migração (com Docker rodando):

```
docker exec -i labprogiiii-db-1 psql -U admin -d rcp_data_imob <
migration_f2.sql
```

14.2.2 Regras de Consistência

O campo `ativa` (já existente) é mantido sincronizado com o `status` automaticamente pela API.

Status	ativa	Descrição
pendente	true	Reserva criada, aguardando confirmação
confirmada	true	Reserva confirmada — bloqueia o imóvel no período
cancelada	false	Reserva cancelada — libera o imóvel

Table 28: Regras de consistência entre status e ativa

14.3 Alterações no Backend

14.3.1 Novos Endpoints em /locacoes

Método	Endpoint	O que há de novo
POST	/locacoes	Aceita campo status; calcula valor_total automaticamente
PUT	/locacoes/:id	NOVO — edição completa com revalidação de disponibilidade
PATCH	/locacoes/:id/status	NOVO — altera apenas o status, revalidando conflito ao confirmar
PATCH	/locacoes/:id/encerrar	Mantido por compatibilidade; agora define status=cancelada
GET	/locacoes	Aceita query param <code>?status=</code> para filtrar por status

Table 29: Endpoints novos/modificados — F2

14.3.2 Lógica de Verificação de Conflito

Conflito de datas é verificado apenas para reservas com `status = confirmada`, excluindo a própria reserva em caso de edição:

```
-- Verifica se existe outra reserva CONFIRMADA que sobrepe o
-- periodo
SELECT 1 FROM locacoes
WHERE imovel_id = $1
      AND id != $2          -- exclui a propria reserva (edicao)
      AND status = 'confirmada'
      AND data_inicio <= $4 -- inicio da nova <= fim da existente
      AND (data_fim IS NULL OR data_fim >= $3)
```

Se conflito detectado, retorna HTTP 409 Conflict com mensagem descritiva.

14.3.3 Cálculo Automático de Valor Total

```
const dias = Math.ceil((new Date(data_fim) - new Date(
  data_inicio)) / 86400000);
const meses = dias / 30;
```

```
const valor_total = parseFloat(valor_mensal) * meses;
```

14.4 Alterações no Frontend

14.4.1 Tela de Locações — Atualizada

Recurso	Descrição
Badges de status	Indicadores visuais coloridos: verde=confirmada, amarelo=pendente, vermelho=cancelada
Filtros por status	Botões no topo com contador de reservas em cada estado (Todas / Pendente / Confirmada / Cancelada)
Ações rápidas por linha	Botao confirmar, cancelar ou reabrir diretamente na tabela
Edição completa	Botão de lápis abre modal com todos os campos editáveis, incluindo status
Valor total calculado	Box verde exibe o valor total estimado em tempo real
Mensal + Total na tabela	A coluna de valor exibe tanto o valor mensal quanto o total calculado

Table 30: Recursos da tela de Locações atualizada

14.4.2 Tela de Agenda — Nova (/calendario)

Recurso	Descrição
Calendário mensal	Grade 7×6 com todos os dias do mês, preenchendo com dias do mês anterior/próximo
Navegação	Botões < e > para navegar entre meses; exibe Mês e Ano no cabeçalho
Eventos coloridos	Cada locação aparece nos dias do seu período com cor de acordo com o status
Popup de detalhes	Clique em qualquer evento abre card com: imóvel, cliente, período, valor mensal e total
Filtro por imóvel	Select no topo filtra o calendário para exibir apenas reservas de um imóvel específico
Legenda	Indicadores de cor explicando cada status, exibidos acima do calendário

Table 31: Recursos da tela de Agenda (Entregável 6)

Cores dos eventos no calendário:

Status	Cor do evento
confirmada	Verde (bg-emerald)
pendente	Amarelo/Âmbar (bg-amber)
cancelada	Vermelho (bg-rose)

Table 32: Cores dos eventos por status

14.4.3 Arquivos Modificados/Criados

Arquivo	Operação	Descrição
frontend/src/pages/Locacoes.jsx	Substituído	Status, filtros, ações rápidas, edição, valor total
frontend/src/pages/Calendario.jsx	Criado	Nova tela de agenda mensal com eventos e popup
frontend/src/App.jsx	Modificado	Adicionada rota /calendario
frontend/src/components/Layout.jsx	Modificado	Adicionado item Agenda no menu lateral
backend/src/routes/locacoes.js	Substituído	CRUD completo com status, edição, conflito e valor_total

Table 33: Arquivos modificados/criados — Entregável 6

14.5 Fluxo Completo da F2

Passo	Ação	Resultado
1	Acessar /locacoes e clicar em Nova Locação	Modal de criação abre com campos de imóvel, cliente, datas, valor e status
2	Preencher datas e valor mensal	Box verde exibe valor total calculado automaticamente em tempo real
3	Definir status como Pendente e salvar	Reserva criada com status=pendente, sem verificação de conflito
4	Clicar em ✓ na linha da reserva	API chama PATCH /locacoes/:id/status com {status:'confirmada'} — verifica conflito
5	Tentar confirmar com período conflitante	API retorna 409 Conflict com mensagem de erro exibida na tela
6	Clicar no lápis para editar	Modal de edição abre pré-preenchido — permite alterar datas, valor e status
7	Acessar /calendario	Calendário exibe eventos coloridos por status para todas as reservas
8	Clicar em um evento no calendário	Popup exibe imóvel, cliente, período, valor mensal e valor total
9	Filtrar por imóvel no calendário	Apenas reservas do imóvel selecionado são exibidas

Table 34: Fluxo completo da Funcionalidade F2

14.6 Resumo dos Endpoints da F2

Método	Endpoint	Body / Params
GET	/locacoes	?status=... (opcional)
POST	/locacoes	{imovel_id, cliente_id, data_inicio, data_fim, valor_mensal, sta
PUT	/locacoes/:id	{todos os campos}
PATCH	/locacoes/:id/status	{status: 'confirmada' 'pendente' 'cancelada'}
PATCH	/locacoes/:id/encerrar	(sem body)

Table 35: *Endpoints da F2*

Exemplo de resposta do POST /locacoes:

```
{
  "id": 5,
  "imovel_id": 1,
  "cliente_id": 2,
  "data_inicio": "2026-05-01",
  "data_fim": "2026-08-01",
  "valor_mensal": "1500.00",
  "status": "pendente",
  "valor_total": "4500.00",
  "ativa": true,
  "criado_em": "2026-04-20T10:00:00.000Z"
}
```

Exemplo de resposta de conflito (409 Conflict):

```
{
  "erro": "Conflito de datas: imovel ja possui reserva
          confirmada neste periodo."
}
```

14.7 Instruções de Instalação e Atualização

```
# Passo 1 - Aplicar a migracao do banco
docker exec -i labprogiiii-db-1 psql -U admin -d rcp_data_imob <
migration_f2.sql

# Passo 2 a 6 - Substituir/criar os arquivos listados na secao
anterior

# Passo 7 - Rebuild dos containers
docker compose up --build -d

# Enviar para o GitHub
git add backend/src/routes/locacoes.js
git add frontend/src/pages/Locacoes.jsx
git add frontend/src/pages/Calendario.jsx
git add frontend/src/App.jsx
git add frontend/src/components/Layout.jsx
git add migration_f2.sql
git commit -m "feat(f2): status de reservas, edicao, calendario
mensal e valor total"
git push
```

15. Funcionalidade F3 — Controle Financeiro com Dashboard Analítico (Entregável 7)

15.1 Visão Geral

O Entregável 7 implementa a Funcionalidade F3 — **Controle Financeiro com Dashboard Analítico**, completando o terceiro módulo do escopo do projeto. A funcionalidade expande o controle financeiro de um simples cadastro de lançamentos para uma plataforma analítica completa, com endpoints de agregação no backend, edição e exclusão de lançamentos, marcação automática de atrasados, filtros avançados e visualizações com gráfico de pizza por status.

15.2 Requisitos Atendidos

Requisito	Status
Endpoints da API necessários para a F3 implementados	Concluído
Persistência no banco funcionando para a F3 (migration aplicada)	Concluído
Telas e componentes do frontend web implementados para a F3	Concluído
Fluxo funcionando do início ao fim no ambiente Docker	Concluído
Subir tudo via <code>docker compose up</code>	Concluído
Executar fluxo completo no navegador, consumindo API e persistindo	Concluído
Documentação correspondente e capturas de tela atualizadas	Concluído

Table 36: Status dos requisitos — Entregável 7 (F3)

15.3 Alterações no Banco de Dados

15.3.1 Migração aplicada à tabela financeiro

Foi adicionada uma coluna opcional de descrição e índices para acelerar os filtros analíticos, sem impacto nos dados existentes. Em seguida, lançamentos vencidos não pagos são marcados como **atrasado**.

Coluna / Índice	Tipo	Padrão	Descrição
descricao	TEXT	NULL	Texto livre para identificar o lançamento (ex: “aluguel maio/2026”, “IPTU”)
idx_financeiro_status	INDEX	—	Acelera filtro por status
idx_financeiro_tipo	INDEX	—	Acelera filtro por tipo
idx_financeiro_vencimento	INDEX	—	Acelera filtro por data
idx_financeiro_locacao	INDEX	—	Acelera filtro por locação

Table 37: Alterações no banco pela migração F3

```
ALTER TABLE financeiro
  ADD COLUMN IF NOT EXISTS descricao TEXT;

UPDATE financeiro
  SET status = 'atrasado'
 WHERE status = 'pendente'
  AND data_vencimento < CURRENT_DATE;
```

```
CREATE INDEX IF NOT EXISTS idx_financeiro_status      ON
  financeiro(status);
CREATE INDEX IF NOT EXISTS idx_financeiro_tipo        ON
  financeiro(tipo);
CREATE INDEX IF NOT EXISTS idx_financeiro_vencimento ON
  financeiro(data_vencimento);
CREATE INDEX IF NOT EXISTS idx_financeiro_locacao     ON
  financeiro(locacao_id);
```

Como executar a migração (com Docker rodando):

```
docker exec -i labprogiiii-db-1 psql -U admin -d rcp_data_imob <
  migration_f3.sql
```

15.4 Alterações no Backend (backend/src/routes/financeiro.js)

15.4.1 Endpoints novos e modificados

Método	Endpoint	O que há de novo
GET	/financeiro	MODIFICADO — aceita filtros via query (tipo, status, locacao_id, data_inicio, data_fim); faz JOIN com imóvel/cliente
GET	/financeiro/resumo	NOVO — KPIs agregados por SQL: total_receitas, total_despesas, saldo, pendente, atrasado, contagens
GET	/financeiro/por-mes	NOVO — série temporal mês a mês (?meses=N, padrão 6)
GET	/financeiro/:id	NOVO — detalhe de um lançamento
POST	/financeiro	MODIFICADO — aceita campo descricao; calcula status inicial conforme vencimento
PUT	/financeiro/:id	NOVO — edição completa com sincronização de data_pagamento ao status
PATCH	/financeiro/:id/pagar	MANTIDO
DELETE	/financeiro/:id	NOVO — remoção de lançamento

Table 38: Endpoints da F3

15.4.2 Auto-update de atrasados

Toda chamada a GET /financeiro e GET /financeiro/resumo dispara antes:

```
UPDATE financeiro
  SET status = 'atrasado'
 WHERE status = 'pendente'
  AND data_vencimento < CURRENT_DATE
```

Garantindo que a UI nunca exiba pendentes vencidos sem flag de atrasado.

15.4.3 Exemplo de resposta — GET /financeiro/resumo

```
{
```

```

    "total_receitas": 18000.00,
    "total_despesas": 4500.00,
    "receitas_recebidas": 12000.00,
    "despesas_pagas": 1500.00,
    "total_pendente": 3000.00,
    "total_atrasado": 1500.00,
    "saldo": 13500.00,
    "qtd_pendente": 2,
    "qtd_atrasado": 1,
    "qtd_pago": 6,
    "qtd_total": 9
  }

```

15.4.4 Exemplo de resposta — GET /financeiro/por-mes?meses=6

```

[
  { "mes": "2025-11", "receita": 1500.00, "despesa": 0 },
  { "mes": "2025-12", "receita": 1500.00, "despesa": 250.00 },
  { "mes": "2026-01", "receita": 3000.00, "despesa": 0 },
  { "mes": "2026-02", "receita": 3000.00, "despesa": 500.00 },
  { "mes": "2026-03", "receita": 4500.00, "despesa": 1750.00 },
  { "mes": "2026-04", "receita": 4500.00, "despesa": 2000.00 }
]

```

15.5 Alterações no Frontend

15.5.1 Tela Financeiro — Reescrita (/financeiro)

Recurso	Descrição
4 Cards analíticos	Total Receitas, Total Despesas, Saldo (verde/vermelho conforme sinal), Pendente+Atrasado com contagens
Gráfico donut	PieChart (Recharts) com distribuição por status (pago / pendente / atrasado), exibindo quantidade de lançamentos em cada estado
Box “Visão Geral”	Mosaico com valores monetários de Recebido, Pago, Pendente, Atrasado, Lançamentos e Saldo
Filtros expandidos	Tipo, status, locação, data início e data fim — todos enviados como query params para o backend
Botão Limpar filtros	Surge automaticamente quando há filtros ativos
Edição inline	Botão de lápis em cada linha abre o modal pré-preenchido para edição completa via PUT
Exclusão com confirmação	Botão de lixeira abre modal de confirmação antes do DELETE
Coluna Descrição	Exibe a descrição opcional do lançamento e a locação relacionada (imóvel + cliente)

Table 39: Recursos da tela Financeiro reescrita (Entregável 7)

15.5.2 Dashboard — Atualizado

A página / (Dashboard) agora consome os endpoints analíticos do backend:

- GET `/financeiro/resumo` substitui o cálculo client-side da Receita do Mês;
- GET `/financeiro/por-mes?meses=6` substitui a agregação manual dos últimos 6 meses para o gráfico de barras Receita vs. Despesa;
- Reduz o payload e a CPU do cliente, e centraliza a lógica analítica no SQL.

15.5.3 Arquivos Criados/Modificados

Arquivo	Operação	Descrição
migration_f3.sql	Criado	Adiciona coluna <code>descricao</code> , marca atrasados, cria índices
backend/src/routes/financeiro.js	Substituído	5 novos endpoints + filtros + auto-update + JOIN com imóvel/cliente
frontend/src/pages/Financeiro.jsx	Substituído	4 cards, donut, filtros, edição/exclusão, modal unificado
frontend/src/pages/Dashboard.jsx	Modificado	Passa a consumir <code>/financeiro/resumo</code> e <code>/financeiro/por-mes</code>
docs/main.tex	Modificado	Nova seção F3, tabela de endpoints atualizada, conclusão
README.md	Modificado	Tabela de endpoints <code>/financeiro</code> expandida

Table 40: Arquivos criados/modificados — Entregável 7

15.6 Fluxo Completo da F3 (end-to-end no Docker)

Passo	Ação	Resultado
1	<code>docker compose up -build -d</code>	db, backend e frontend sobem; healthcheck do PostgreSQL libera o backend
2	Aplicar a migração F3	Coluna <code>descricao</code> , índices criados e atrasados sincronizados
3	Acessar <code>http://localhost:5173/financeiro</code>	Frontend chama <code>/financeiro/resumo</code> ; cards e gráfico renderizam
4	Clicar em “Novo Lançamento” e preencher	<code>POST /financeiro</code> grava no banco com descrição opcional
5	Aplicar filtro por <code>status=atrasado</code>	Frontend repete o GET com query params; lista re-renderiza
6	Clicar no lápis de uma linha e editar	<code>PUT /financeiro/:id</code> persiste alterações
7	Clicar em “Pagar”	<code>PATCH /financeiro/:id/pagar</code> grava <code>data_pagamento</code>
8	Clicar na lixeira e confirmar	<code>DELETE /financeiro/:id</code> remove do banco; lista atualiza
9	Voltar ao Dashboard	KPIs e gráfico já refletem as mudanças via endpoints analíticos

Table 41: Fluxo end-to-end da Funcionalidade F3 no ambiente Docker

15.7 Resumo dos Endpoints F3

Método	Endpoint	Retorno
GET	<code>/financeiro?...</code>	Lista de lançamentos com JOIN de imóvel/cliente
GET	<code>/financeiro/resumo</code>	Objeto com KPIs analíticos consolidados
GET	<code>/financeiro/por-mes?meses=6</code>	Array de {mes, receita, despesa}
GET	<code>/financeiro/:id</code>	Objeto único do lançamento
POST	<code>/financeiro</code>	201 — lançamento criado
PUT	<code>/financeiro/:id</code>	200 — atualizado
PATCH	<code>/financeiro/:id/pagar</code>	200 — status=pago, data_pagamento=hoje
DELETE	<code>/financeiro/:id</code>	200 — { mensagem, registro }

Table 42: Resumo de endpoints da F3

16. Execução do Ambiente

O projeto foi preparado para execução com Docker Compose, permitindo subir todos os serviços de forma integrada.

```
# Subir todos os servicos (build + start em background)
docker compose up --build -d

# Verificar status dos containers
docker compose ps

# Ver logs de um servico especifico
docker compose logs frontend
docker compose logs backend

# Parar os servicos
docker compose down

# Remover tambem os volumes (apaga dados do banco)
docker compose down -v
```

16.1 Acessos

Serviço	URL
Frontend	http://localhost:5173
API REST	http://localhost:3000
PostgreSQL	localhost:5432

Table 43: URLs de acesso aos serviços

16.2 Telas Disponíveis

Rota	Tela
/	Dashboard — KPIs e gráfico financeiro
/imoveis	Gestão de Imóveis
/clientes	Gestão de Clientes
/locacoes	Gestão de Locações (com status F2)
/financeiro	Controle Financeiro
/disponibilidade	Busca de Disponibilidade e Reserva (F1)
/calendario	Agenda Mensal com Calendário (F2)

Table 44: Telas disponíveis no sistema

17. Considerações sobre a Implementação

A proposta do sistema demonstra uma evolução consistente ao longo dos entregáveis:

- No **Entregável 0**, foi definida a proposta do sistema, seu escopo funcional, a stack e a visão geral das telas;
- No **Entregável 2**, foi consolidada a implementação da API REST, com organização modular, integração com PostgreSQL e endpoints CRUD por entidade;
- No **Entregável 3**, foi configurado o banco de dados PostgreSQL via Docker, com modelagem completa das entidades, inicialização automática do schema e validação de CRUD via Postman;
- No **Entregável 4**, foi implementado o frontend completo com React, Vite e Tailwind CSS, configurado no Docker com proxy para o backend, consumindo dados reais da API em todas as 5 telas do sistema;
- No **Entregável 5**, foi implementada a Funcionalidade F1 completa (Busca de Disponibilidade e Criação de Reserva), com novo endpoint de API com lógica de conflito de datas, nova tela no frontend e fluxo de reserva end-to-end funcionando no Docker;
- No **Entregável 6**, foi implementada a Funcionalidade F2 completa (Sistema de Reservas com Status), com ciclo de vida completo de reservas, calendário mensal interativo, cálculo automático de valor total e edição com revalidação de conflito;
- No **Entregável 7**, foi implementada a Funcionalidade F3 completa (Controle Financeiro com Dashboard Analítico), com endpoints de agregação no backend, filtros avançados, edição e exclusão de lançamentos, marcação automática de atrasados, gráfico de pizza por status e Dashboard alimentado por SQL agregado.

18. Conclusão

O Sistema de Gestão de Locações de Imóveis (RCP Data Imob) foi projetado e implementado como uma solução web completa para atender às necessidades de uma administradora de imóveis. O sistema integra cadastro, locações, categorização e controle financeiro em uma única aplicação com interface visual profissional.

Do ponto de vista técnico, a solução utiliza tecnologias atuais e amplamente adotadas no desenvolvimento web, com separação clara entre frontend, backend e banco de dados. A utilização de Docker Compose contribui para a reprodutibilidade do ambiente, enquanto a API REST organizada por entidade favorece escalabilidade e manutenção.

O frontend implementado oferece uma experiência de usuário completa, com Dashboard analítico, CRUD funcional em todas as entidades, design responsivo com Tailwind CSS, gráficos interativos com Recharts, e micro-interações que conferem profissionalismo à interface.

O Entregável 6 finaliza a Funcionalidade F2, completando o ciclo de vida das reservas com controle de status, calendário mensal de ocupações, cálculo automático de valor total e validação rigorosa de conflitos de datas.

O Entregável 7 conclui o terceiro pilar do escopo, entregando a Funcionalidade F3 (Controle Financeiro com Dashboard Analítico). A API ganhou endpoints de agregação (`/financeiro/resumo` e `/financeiro/por-mes`), além de filtros, edição (PUT), exclusão (DELETE) e marcação automática de lançamentos vencidos como atrasados. A tela de Financeiro foi reescrita com 4 KPIs, gráfico de pizza por status, filtros expandidos e CRUD

completo via modal. O Dashboard agora consome a agregação direto do SQL, eliminando processamento no cliente. Com isso, todos os três módulos do escopo (F1, F2 e F3) estão operando ponta-a-ponta no ambiente Docker.

19. Referências Internas do Projeto

- Entregável 0 — Definição do tema, escopo, stack e telas;
- Entregável 2 — Implementação da API REST e organização do backend;
- Entregável 3 — Banco de dados PostgreSQL + Docker + testes CRUD via Postman;
- Entregável 4 — Frontend React + Vite + Tailwind CSS no Docker, consumindo API;
- Entregável 5 — Funcionalidade F1: Busca de Disponibilidade e Criação de Reserva;
- Entregável 6 — Funcionalidade F2: Sistema de Reservas com Status e Calendário Mensal;
- Entregável 7 — Funcionalidade F3: Controle Financeiro com Dashboard Analítico;
- Coleção Postman — `postman/RCP_Data_Imob.postman_collection.json`.

Marcell Parra Araújo B. Silva • Rafael Vargas Carvalheira • Ruan Pablo Rodrigues

Março de 2026 — atualizado em Abril de 2026 (Entregável 7)